Guía de Recursividad en Python

Explicación teórica, ejercicios prácticos y traza de ejecución para presentación académica

¿Qué es la Recursividad?

La recursividad es una técnica de programación donde una función se llama a sí misma para resolver un problema dividiéndolo en subproblemas más pequeños. Todo algoritmo recursivo requiere dos componentes indispensables:

- **Caso Base:** La condición de parada que evita un bucle infinito.
- **Paso Recursivo:** La llamada a la misma función pero con una entrada reducida hacia el caso base.

1. Factorial de un número: `factorial(n)`

Calcula el producto de todos los enteros positivos desde 1 hasta n. Por ejemplo, `5! = 5 × 4 × 3 × 2 × 1 = 120`.

```
def factorial(n):
    # Caso base: el factorial de 0 o 1 siempre es 1
    if n <= 1:
        return 1
    # Paso recursivo: n multiplicado por el factorial de (n - 1)
    return n * factorial(n - 1)
```

Aclaración clave sobre el parámetro n: n es una variable que representa cualquier número de entrada. No tiene un valor fijo hasta que llamamos a la función (ej. `factorial(5)`).

2. Filtrar pares y elevar al cuadrado: `filtrar_y_cuadrado(lista)`

Sustituye `map` y `filter` procesando la lista en un solo paso recursivo mediante concatenación de listas (operador +).

```
def filtrar_y_cuadrado(lista):
    # Caso base: lista vacía
    if not lista:
        return []

    primer = lista[0]
    resto = lista[1:]

    # Si es par, lo elevamos al cuadrado y concatenamos con el resto
    if primer % 2 == 0:
        return [primer ** 2] + filtrar_y_cuadrado(resto)
    else:
        # Si es impar, lo ignoramos y procesamos el resto
        return filtrar_y_cuadrado(resto)
```

> **Diferencia entre sumar y concatenar listas (Operador +)**
>
> En la función `filtrar_y_cuadrado`, el uso de + no suma valores numéricos, sino que **une (concatena) listas**. Por ejemplo: `[4] + [16, 36]` da como resultado `[4, 16, 36]`.

3. Suma total de una lista: `suma_recursiva(lista)`

Calcula la suma acumulada de todos los elementos presentes en una lista dada.

```
def suma_recursiva(lista):
    # Caso base: una lista vacía suma 0
    if not lista:
        return 0
    # Paso recursivo: primer elemento + suma recursiva del resto
    return lista[0] + suma_recursiva(lista[1:])
```

4. Invertir una cadena: `invertir_cadena(texto)`

Recibe una cadena de caracteres y devuelve el texto invertido invirtiendo los sub-strings.

```
def invertir_cadena(texto):
    # Caso base: textos de longitud 0 o 1 ya están invertidos
    if len(texto) <= 1:
        return texto
    # Paso recursivo: último carácter + inversión del texto restante
    return texto[-1] + invertir_cadena(texto[:-1])
```

5. Encontrar el número máximo: `maximo_recursivo(lista)`

Sustituye la lógica de `max()` o acumuladores comparando elementos uno a uno.

```
def maximo_recursivo(lista):
    # Caso base: si solo queda un elemento, ese es el máximo de esa sublista
    if len(lista) == 1:
        return lista[0]

    max_resto = maximo_recursivo(lista[1:])

    # Comparación entre el elemento actual y el máximo del resto
    return lista[0] if lista[0] > max_resto else max_resto
```

Script Completo Ejecutable (Un solo archivo)

A continuación se presenta el código integrado listo para ejecutar con pruebas de demostración:

```
# ============================================================================
# SCRIPT DE DEMOSTRACIÓN COMPLETO
# ============================================================================

if __name__ == "__main__":
    print("=== PRUEBAS DE FUNCIONES RECURSIVAS ===")

    # 1. Factorial
    print("1. Factorial(5):", factorial(5))

    # 2. Filtrar pares y elevar al cuadrado
    print("2. Pares al cuadrado [1..6]:", filtrar_y_cuadrado([1, 2, 3, 4, 5, 6]))

    # 3. Suma recursiva
    print("3. Suma [5, 10, 15]:", suma_recursiva([5, 10, 15]))

    # 4. Invertir cadena
    print("4. Cadena 'Python' invertida:", invertir_cadena("Python"))

    # 5. Máximo elemento
    print("5. Máximo en [3, 17, 8, 25, 4]:", maximo_recursivo([3, 17, 8, 25, 4]))
```

Pila de Llamadas (Call Stack) para Explicar al Profesor

Fase de Apilado (Push)

Cada vez que la función se llama a sí misma, Python guarda el estado actual en la memoria (pila de llamadas) y espera a que la sub-llamada responda.

Fase de Desapilado (Pop)

Al llegar al caso base, la pila comienza a resolverse de abajo hacia arriba, multiplicando, sumando o concatenando los valores retenidos.

Ejemplo de Traza en Memoria para `factorial(3)`:

- `factorial(3)` → retorna `3 * factorial(2)` (En espera)
- `factorial(2)` → retorna `2 * factorial(1)` (En espera)
- `factorial(1)` → retorna `1` (Caso base alcanzado)
- Retorno: `2 * 1 = 2` → luego `3 * 2 = 6`. Resultado final: 6.